Name:

PRN:

Experiment no 6

Assignment 6: Introduction to MPI : Installation, Processes, Ranks, Groups and Point-to-Point Communication

Aim:

To configure the MPI environment and understand the fundamentals of distributed-memory parallel programming by creating MPI processes, identifying process ranks, working with MPI groups, and implementing one-to-one communication using MPI_Send() and MPI_Recv().

Why Are We Learning This?

OpenMP allows multiple threads to work in a shared memory space. However, many HPC applications run on clusters where each computing node has its own memory.

MPI (Message Passing Interface) provides a standard programming model for such distributed-memory systems.

In MPI:

- A program is executed by multiple processes.
- Each process has its own address space.
- Processes communicate by exchanging messages.
- Each process is identified by a unique rank.
- Processes can be organized into groups and communicators.

MPI is widely used in:

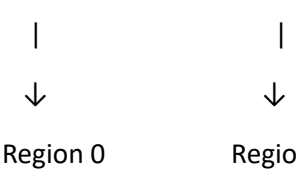
- Weather and climate simulation
- Computational fluid dynamics
- Molecular dynamics
- Large-scale scientific simulations
- Distributed AI/ML workloads
- Big-data processing
- Supercomputing applications

HPC in Practice

Consider a weather simulation running on 100 computing nodes.

Each process may calculate the weather conditions for a particular geographical region. However, neighbouring regions need to exchange information.

For example:



MPI provides the communication mechanism required to exchange this information.

If each process has its own memory:

How can Process 0 send data to Process 1?

The answer is message passing.

This assignment introduces the basic mechanism using:

MPI_Send()

MPI_Recv()

Learning Focus

This assignment introduces:

- MPI environment
- MPI program structure
- MPI initialization and termination
- MPI processes
- MPI rank
- MPI size
- MPI communicator
- MPI groups
- Point-to-point communication
- MPI_Send()
- MPI_Recv()
- Message tags

- Source and destination
- Basic performance measurement

Pre-Lab Activity

Answer the following questions before starting the assignment.

1. What is distributed-memory parallel computing?
2. What is MPI?
3. What is the difference between a process and a thread?
4. What is an MPI rank?
5. What is an MPI communicator?
6. What is the purpose of MPI_COMM_WORLD?
7. What is the difference between MPI_Send() and MPI_Recv()?
8. Why is message passing necessary in distributed-memory systems?

Section A – Basics

Task 1: MPI Installation and Environment Verification

Install and configure an MPI implementation such as **OpenMPI** or **MPICH**.

Verify the installation using appropriate commands.

For example:

mpicc --version

and:

mpirun --version

or:

mpiexec --version

Record:

- MPI implementation
- MPI version
- C compiler version
- Operating system
- CPU configuration

Task 2: First MPI Program

Write an MPI program to display:

Hello from MPI

The program must contain the appropriate MPI initialization and termination routines.

Identify the purpose of:

MPI_Init()

MPI_Finalize()

Compile and execute the program using:

mpicc hello.c -o hello

mpirun -np 4 ./hello

Task 3: Understanding MPI Rank and Size

Modify the program to display:

- Process rank
- Total number of processes

Use:

MPI_Comm_rank()

MPI_Comm_size()

Execute using:

- 2 processes
- 4 processes
- 8 processes

Expected Output

The order of output may vary.

Process 0 of 4

Process 1 of 4

Process 2 of 4

Process 3 of 4

Record Your Observation

Number of Processes	Observed Ranks	Total Processes
2		
4		
8		

Task 4: Understanding MPI Groups and Communicators

Investigate the relationship between:

MPI_COMM_WORLD → MPI Group → MPI Communicator

Using MPI group-related functions, create a subgroup containing selected processes from MPI_COMM_WORLD.

For example, if there are 8 processes:

MPI_COMM_WORLD

Rank: 0 1 2 3 4 5 6 7

↓ ↓ ↓ ↓

Selected: 0 2 4 6

Create a subgroup containing selected ranks and explore:

- MPI_Comm_group()
- MPI_Group_incl()
- MPI_Comm_create_group()

Display the rank of each process in the newly created communicator.

Important

Understand the difference between:

Global rank

and

Rank within a communicator.

Think Like an HPC Programmer

Consider 16 processes running on an HPC cluster.

Suppose:

- Processes 0–7 perform one computational task.
- Processes 8–15 perform another computational task.

Answer:

1. Why might it be useful to create separate communicators?
2. Can the same process belong to different communication groups?
3. Why is the communicator important in MPI communication?
4. How can groups help organize large parallel applications?

Section B – Application: One-to-One Communication

Task 5: MPI Send and Receive

Implement a program in which **Process 0 sends a message to Process 1**.

Process 1 should receive the message and display it.

Use:

MPI_Send()

MPI_Recv()

Task 6: Integer Data Communication

Modify the previous program so that Process 0 sends an integer value to Process 1.

Example:

Process 0:

Value = 100

MPI_Send()

↓

Process 1:

Received Value = 100

Verify the received value.

Task 7: Array Communication

Extend the program so that Process 0 sends an array of integers to Process 1.

For example:

Process 0

10 20 30 40 50

|

| MPI_Send()

↓

Process 1

10 20 30 40 50

Process 1 should calculate the:

- Sum
- Average
- Maximum

of the received array.

Performance Activity

Measure the communication time for different message sizes.

Test with:

- 1 KB
- 10 KB
- 100 KB
- 1 MB
- 10 MB

Record:

Message Size	Communication Time
1 KB	

Message Size	Communication Time
10 KB	
100 KB	
1 MB	
10 MB	

Analyze

1. How does communication time change with message size?
2. Why is communication overhead important in distributed-memory computing?
3. Why might MPI communication become a bottleneck?

Challenge: Communication

Implement a **communication** between Process 0 and Process 1.

Requirement

Process 0 sends a message to Process 1.

Process 1 sends the message back.

This continues for a specified number of iterations.

Process 0 Process 1

Send ---------->

<---------- Receive

Send ---------->

<---------- Receive

...

Measure the total communication time.

Calculate:

Round-Trip Time

$$RTT = \frac{Total\ Time}{Number\ of\ Iterations}$$

Investigate how RTT changes with message size.

Advanced Challenge: Avoiding Communication Problems

Modify the program to investigate the behavior of blocking communication.

Consider:

MPI_Send()

MPI_Recv()

Study what happens if both processes attempt to send data before receiving.

Explain:

- Why deadlock can occur.
- How message ordering affects communication.
- How communication can be redesigned to avoid deadlock.

Do not deliberately run an unbounded deadlocking program on a shared HPC system. Use a controlled small test and terminate it safely if required.

HPC / AI Case Study

Choose **one** application:

- Weather forecasting
- Computational fluid dynamics
- Molecular dynamics
- Distributed deep learning
- Large-scale scientific simulation
- Graph processing
- Image processing

Identify:

1. Why is distributed computing required?
2. What data needs to be exchanged between processes?
3. Which processes need to communicate?
4. How could MPI be used?
5. What would happen if communication time becomes larger than computation time?

Write a **300–500 word case study**.

Questions

Basic

1. What is MPI?
2. What is an MPI process?
3. What is an MPI rank?
4. What is the purpose of MPI_COMM_WORLD?
5. What is the difference between MPI_Send() and MPI_Recv()?

Moderate

6. Differentiate between an MPI group and communicator.
7. Why can two processes have different ranks in different communicators?
8. What information is specified when using MPI_Send()?
9. What information is specified when using MPI_Recv()?
10. Why does the order of send and receive operations matter?

Advanced

11. What is communication overhead?
12. Explain how message size affects MPI performance.
13. What is deadlock in MPI?
14. Why can blocking communication lead to deadlock?
15. Why is communication an important consideration when designing distributed-memory algorithms?

Performance Analysis

For the ping-pong experiment, prepare:

Message Size	Iterations	Total Time	Average RTT
1 KB			
10 KB			
100 KB			
1 MB			
10 MB			

Plot:

1. **Message Size vs Communication Time**
2. **Message Size vs Round-Trip Time**

Discuss the observed trend.

Answer the following:

1. What is the fundamental difference between OpenMP and MPI?
2. When would you choose MPI instead of OpenMP?
3. Why is data distribution important in MPI?
4. What is the role of communication in distributed-memory applications?
5. What challenges arise when the number of MPI processes increases?
6. How could the concepts learned in this assignment be used in the next MPI assignment involving matrix computations?

Deliverables

Submit:

1. MPI installation and environment details.
2. Source code for all completed tasks.
3. Execution outputs.
4. MPI rank/group/communicator observations.
5. Performance measurements.
6. Communication-time graphs.
7. Answers to discussion questions.
8. AI/HPC case study.
9. Brief conclusion.